Estructura de Datos

Proyecto 2: RPG por terminal

Docente: Jacqueline Aldridge

Integrantes:
Pablo Serón
Daniela Soto
Carlos Cofré

Fecha de entrega: 30 de octubre de 2026



Índice

1. Introducción **2**

2. Visión general del diseño **2**

3. Lista enlazada: Inventario **4**
3.1. Justificación de la estructura 4
3.2. Representación 4
3.3. Operaciones implementadas 4
3.4. Uso en el juego 5
3.5. Principios del tipo de dato abstracto 5
3.6. Manejo de casos límite 5
3.7. Limitaciones conocidas 5

4. Pila: Historial y deshacer **6**
4.1. Justificación de la estructura 6
4.2. Representación 6
4.3. Operaciones implementadas 6
4.4. Uso en el juego 6
4.5. Principios del tipo de dato abstracto 7
4.6. Manejo de casos límite 7
4.7. Limitaciones conocidas 7

5. Cola: Enemigos y combate **8**
5.1. Justificación cola 8
5.2. Combate 8
5.3. Representación y operaciones 8
5.4. Principios del tipo de dato abstracto 8
5.5. Manejo de casos límite 9

6. Guardar y cargar partida **10**

7. Ejemplos de ejecución **11**
7.1. Exploración y combate en la Catacumba 11
7.2. Cola de tres enemigos, equipamiento y Bengala (Páramo) 11
7.3. Inventario, historial, deshacer y guardado 11

8. Pruebas **12**

9. Conclusiones **12**

10. Contribución por integrante **13**

11. Declaración de uso de IA **13**

1. Introducción

Este proyecto consiste en una versión simplificada de un juego de rol por terminal. El personaje explora zonas, recoge objetos, administra su inventario y combate contra enemigos. Todo el sistema se construye sobre tres estructuras de datos implementadas por el grupo, sin bibliotecas que las resuelvan: una **lista enlazada** (inventario), una **pila** (historial y deshacer) y una **cola** (enemigos en combate).

El objetivo de este informe es explicar qué estructura se eligió para cada funcionalidad y por qué, cómo se implementó cada operación, cómo se respetan los principios de cada tipo de dato abstracto y cómo se manejan los casos límite (estructura vacía, un solo elemento, memoria insuficiente y eliminación de elementos).

El código se organiza en tres carpetas: `inc/` (archivos de cabecera `.h`), `src/` (implementaciones `.c`, incluido `main.c`) y `test/` (programas de prueba). El `Makefile` compila el juego con `make` y ejecuta las pruebas con `make test`. Cada módulo es un par `.h/.c` con su autor identificado y sus funciones documentadas (propósito, parámetros y valores de retorno). El trabajo se coordinó con Git y GitHub, con commits de los tres integrantes, siguiendo las reglas del `README_Organizacion.md`.

2. Visión general del diseño

La Tabla 1 resume qué estructura resuelve cada funcionalidad y quién es el responsable principal.

Estructura	**Funcionalidad**	**Módulos**	**Responsable**
Lista enlazada simple	Inventario	`objeto`, `inventario`, `uso_objeto`, `guardado`	Pablo Serón
Pila	Historial y deshacer	`historial`, `personaje`	Carlos Cofré
Cola	Enemigos y combate	`cola_enemigo`, `enemigo`, `combate`	Daniela Soto

Cuadro 1: Estructuras de datos del proyecto.

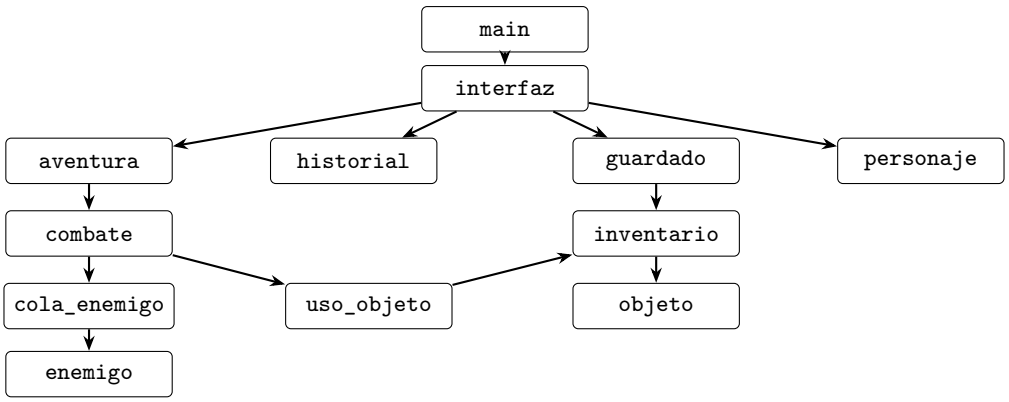


Figura 1: Dependencias principales entre módulos (una flecha significa “usa a”).

El punto de entrada es `main.c`, que solo llama a `iniciar_juego()` del módulo `interfaz`. La interfaz crea el inventario, el historial y el personaje, muestra el menú principal y delega en el resto: `aventura` para explorar, `inventario` y `uso_objeto` para administrar objetos, `historial` para registrar y deshacer acciones y `guardado` para guardar y cargar la partida. La aventura

arma la cola de enemigos de cada zona y llama a `combate`, que atiende a los enemigos en orden y usa `uso_objeto` para equipar o consumir objetos durante la pelea.

3. Lista enlazada: Inventario

3.1. Justificación de la estructura

El inventario es una colección que crece y decrece durante la partida: se agregan objetos al recogerlos y se eliminan al consumirlos o descartarlos. Se eligió una **lista enlazada simple con nodo centinela** por estas razones:

- **Frente a un arreglo:** no tiene un tamaño máximo fijo y eliminar un objeto del medio no obliga a desplazar los demás elementos, solo a reconectar dos punteros.
- **Frente a una lista doblemente enlazada:** el inventario solo se recorre hacia adelante (listar, buscar, eliminar), por lo que el puntero al nodo anterior no se necesita y se ahorra memoria y complejidad.
- **Frente a una pila o una cola:** el jugador debe poder buscar, usar o eliminar un objeto cualquiera por su ID o nombre, no solo el primero o el último, y esos acceso no son operaciones de esas estructuras.
- **Nodo centinela:** es un nodo falso que siempre existe al inicio de la lista. Gracias a él, insertar o eliminar el primer objeto se hace exactamente igual que con cualquier otro, sin un caso especial para la cabeza.

3.2. Representación

```c
typedef struct _NodoInventario {
    Objeto objeto; /* copia del objeto */
    struct _NodoInventario *siguiente;
} NodoInventario;

typedef struct _Inventario {
    NodoInventario *cabeza; /* apunta al centinela */
    int cantidad; /* objetos reales (sin centinela) */
} Inventario;
```

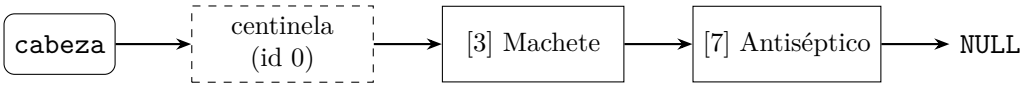


Figura 2: Inventario con dos objetos. El centinela (línea punteada) no cuenta como objeto.

3.3. Operaciones implementadas

Operación	Descripción	Costo
`inventario_inicializar`	Crea el centinela y deja la lista vacía.	$O(1)$
`inventario_agregar`	Copia el objeto al final; rechaza IDs repetidos.	$O(n)$
`inventario_listar`	Imprime todos los objetos.	$O(n)$
`inventario_buscar_por_id` / `_por_nombre`	Devuelve un puntero al objeto o `NULL`.	$O(n)$
`inventario_eliminar_por_id` / `_por_nombre`	Elimina un nodo y devuelve una copia del objeto.	$O(n)$
`inventario_liberar`	Libera todos los nodos, incluido el centinela.	$O(n)$

Cuadro 2: Operaciones del inventario y su costo, con n la cantidad de objetos.

Para eliminar, se recorre la lista hasta quedar parado en el nodo *anterior* al buscado; luego se guarda el nodo a eliminar, se reconecta el anterior con el siguiente y recién entonces se libera la memoria. La función `eliminar_siguiente_de` concentra esta lógica y es usada tanto por la eliminación por ID como por nombre.

3.4. Uso en el juego

El inventario participa directamente en las funcionalidades pedidas:

- **Recoger objetos** durante la exploración (`inventario_agregar`).
- **Usar o equipar** (`usar_objeto`): las armas y armaduras reemplazan el bono de ataque o defensa y permanecen en el inventario; las pociones curan hasta la vida máxima y se eliminan; los objetos de tipo "otro" (por ejemplo la Bengala) se eliminan y hacen que el enemigo pierda su próximo ataque.
- **Descartar** un objeto por ID o por nombre, devolviendo una copia del objeto eliminado para que el historial pueda registrarlo y deshacerlo.
- **Guardar y cargar** la partida (Sección 6).

3.5. Principios del tipo de dato abstracto

- El resto del programa manipula el inventario solo a través de las funciones de `inventario.h`, sin enlazar nodos a mano.
- Cada nodo guarda una *copia* del objeto, por lo que el inventario es dueño de su memoria y el llamador no necesita liberarla.
- Las funciones de búsqueda devuelven un puntero *dentro* de la lista; ese puntero deja de ser válido si el objeto se elimina. Por eso `consumir_pocion` guarda una copia antes de eliminar.
- Los IDs son únicos: la propia estructura impide insertar duplicados.

3.6. Manejo de casos límite

Caso	**Tratamiento**
Estructura vacía	Listar muestra "El inventario está vacío"; buscar devuelve `NULL`; eliminar devuelve `INVENTARIO_ERROR_NO_ENCONTRADO`.
Un solo elemento	Gracias al centinela, eliminarlo es igual que eliminar cualquier otro: el centinela vuelve a apuntar a `NULL`.
Memoria insuficiente	Si `malloc` falla al crear el centinela o un nodo, se devuelve `INVENTARIO_ERROR_MEMORIA` y el inventario queda sin cambios (la cantidad solo aumenta si todo salió bien).
Eliminación	Se valida que el objeto exista, se reconectan los punteros antes del `free` y se decrementa la cantidad.
Liberación	Se guarda el siguiente nodo antes de liberar cada uno, y la cabeza queda en `NULL`, por lo que liberar dos veces no provoca doble `free`.
Parámetros inválidos	Punteros `NULL` o inventario sin inicializar devuelven `INVENTARIO_ERROR_PARAMETRO`.

Cuadro 3: Casos límite del inventario.

3.7. Limitaciones conocidas

Agregar al final cuesta $O(n)$ porque la lista no mantiene un puntero al último nodo; para un inventario pequeño esto es aceptable. Además, al deshacer un descarte el objeto vuelve al final de la lista y no a su posición original.

4. Pila: Historial y deshacer

4.1. Justificación de la estructura

El historial debe mostrarse desde la acción más reciente a la más antigua y solo permite deshacer la *última* acción. Son exactamente los accesos de una pila (LIFO): apilar al registrar una acción y consultar o sacar solo la cima. Un arreglo o una lista permitirían acceder a acciones intermedias, algo que el juego no necesita; una cola, en cambio, mostraría las acciones de la más antigua a la más reciente y deshacería la primera en vez de la última. Se implementó con nodos enlazados, de modo que crece sin un límite fijo y apilar y desapilar cuestan $O(1)$.

4.2. Representación

```
typedef struct _accion {
    TipoAccion tipo; /* exploracion, descartar, usar, ... */
    char descripcion[LARGO_DESC_ACCION];
    Objeto objeto_involucrado; /* copia, para poder deshacer */
} Accion;

typedef struct _nodo_historial {
    Accion accion;
    struct _nodo_historial *siguiente;
} NodoHistorial;

typedef struct _pila_historial {
    NodoHistorial *cima; /* NULL si la pila esta vacia */
} PilaHistorial;
```

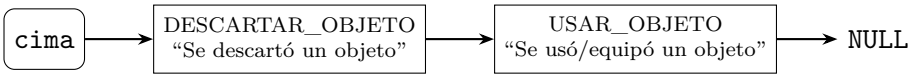


Figura 3: Pila del historial con dos acciones. La más reciente está en la cima.

4.3. Operaciones implementadas

Operación	Descripción	Costo
`historial_crear`	Crea la pila vacía (`cima = NULL`).	$O(1)$
`historial_registrar` (push)	Crea un nodo con el tipo, la descripción y una copia del objeto, y lo deja en la cima.	$O(1)$
`historial_desapilar` (pop)	Quita la cima y devuelve la acción; devuelve `ACCION_VACIA` si no hay nada.	$O(1)$
`historial_cima` (peek)	Devuelve un puntero constante a la acción de la cima sin quitarla (`NULL` si está vacía).	$O(1)$
`historial_esta_vacio`	Indica si la pila no tiene acciones.	$O(1)$
`historial_mostrar`	Imprime de la más reciente a la más antigua.	$O(n)$
`historial_liberar`	Libera todos los nodos y la estructura.	$O(n)$

Cuadro 4: Operaciones del historial, con n la cantidad de acciones.

4.4. Uso en el juego

La interfaz registra una acción cada vez que el jugador explora una zona, usa o equipa un objeto, descarta un objeto o guarda la partida. Para **deshacer** (opción 7 del menú) se consulta

la cima con `historial_cima` sin sacarla: si la pila está vacía se avisa que no hay acciones; si la última acción no es `ACCION_DESCARTAR_OBJETO` se avisa que no se puede deshacer; y si lo es, se reinserta con `inventario_agregar` la copia del objeto guardada en la acción y solo entonces se desapila. Si la reinserción falla (por ejemplo, por falta de memoria) la acción permanece en la pila.

4.5. Principios del tipo de dato abstracto

Solo se accede por la cima: no existen operaciones para leer o modificar nodos intermedios. `historial_cima` y `historial_mostrar` reciben la pila como `const`, por lo que consultar no puede alterarla. Cada acción guarda una *copia* del objeto, así que sigue siendo válida aunque el objeto ya no esté en el inventario.

4.6. Manejo de casos límite

Caso	**Tratamiento**
Pila vacía	`mostrar` imprime "El historial está vacío"; `desapilar` devuelve `ACCION_VACIA`; `cima` devuelve `NULL`; el deshacer informa que no hay acciones.
Un solo elemento	Al desapilarlo, `cima` pasa al `siguiente` del nodo, que es `NULL`, y la pila queda vacía y reutilizable.
Memoria insuficiente	`historial_crear` devuelve `NULL` y avisa por `stderr`; si `malloc` falla en `registrar`, se avisa y la pila queda sin cambios. Todas las operaciones toleran una pila `NULL`.
Eliminación	`desapilar` guarda el nodo, mueve la cima y recién entonces hace `free`.
Liberación	Se guarda el siguiente nodo antes de cada `free` y al final se libera la estructura.

Cuadro 5: Casos límite del historial.

4.7. Limitaciones conocidas

`historial_registrar` no devuelve un código de error: si `malloc` falla solo avisa por pantalla. Además, actualmente se registran la exploración, el uso de objetos, los descartes y el guardado, pero no los eventos dentro del combate (derrotas de enemigos, objetos encontrados). Por último, al deshacer un descarte el objeto vuelve al final del inventario y no a su posición original.

5. Cola: Enemigos y combate

5.1. Justificación cola

La implementación de una cola para los enemigos sirve para manejar de manera secuencial los encuentros en combate por turno. Al almacenar los enemigos de esta manera se consigue que el enemigo en el frente (front) de la cola sea el objetivo de ataque del personaje, mientras que el resto de enemigos esperan su turno por orden de llegada. Una vez que el personaje logra derrotar al enemigo se llama la funcion cola_enemigo_dequeue donde se elimina el enemigo y automaticamente el siguiente enemigo pasa al frente para su turno del combate.

5.2. Combate

El sistema de combate esta diseñado para la interacción del personaje y enemigo. Su flujo principal se modela de la siguiente manera:

- **Menu del combate:** muestra en la terminal los puntos de vida del personaje junto a su ataque (considerando bonificaciones), también los puntos de vida y el nombre del enemigo y por último las opciones disponibles para el jugador.
- **Acciones del jugador:** el combate se desarrolla a través de un switch donde el jugador puede elegir atacar directamente y posteriormente ataca el enemigo y también consultar el inventario para utilizar un objeto ingresando su id.
- **Acciones del enemigo:** cuando le toca el turno al enemigo, el codigo calcula el daño restando la defensa del personaje con el ataque del enemigo. Si la defensa del personaje es mayor al ataque del enemigo se bloquea por completo, en cambio si la defensa es menor el daño recibido va ser la diferencia de estos.

5.3. Representación y operaciones

```c
typedef struct _NodoEnemigo {
    Enemigo enemigo; /* copia del enemigo */
    struct _NodoEnemigo *siguiente;
} NodoEnemigo;

struct _ColaEnemigo {
    NodoEnemigo *front; /* primero en salir */
    NodoEnemigo *rear; /* ultimo en entrar */
    int tamanio;
};
```

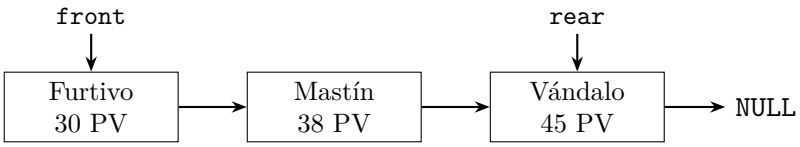


Figura 4: Cola de enemigos de la zona Páramo. El Furtivo se enfrenta primero.

5.4. Principios del tipo de dato abstracto

Se respeta FIFO: los enemigos solo se insertan por el final y solo salen por el frente, por lo que se enfrentan en su orden de aparición. Como `rear` apunta al último nodo, encolar cuesta $O(1)$. Una decisión de diseño es que `cola_enemigo_front` devuelve un *puntero* y no una copia: así el daño que recibe el enemigo durante el combate se conserva de un turno al siguiente.

Operación	Descripción	Costo
`cola_enemigo_create`	Crea la cola vacía.	$O(1)$
`cola_enemigo_enqueue`	Agrega un enemigo al final (usa `rear`); devuelve 1 si tuvo éxito y 0 si no.	$O(1)$
`cola_enemigo_dequeue`	Quita y devuelve el enemigo del frente.	$O(1)$
`cola_enemigo_front`	Devuelve un puntero al enemigo del frente, sin sacarlo.	$O(1)$
`cola_enemigo_is_empty` / `size`	Indican si la cola está vacía y cuántos enemigos tiene.	$O(1)$
`cola_enemigo_make_empty` / `delete`	Liberan todos los nodos (`delete` también la estructura).	$O(n)$

Cuadro 6: Operaciones de la cola de enemigos.

5.5. Manejo de casos límite

Caso	Tratamiento
Cola vacía	`is_empty` devuelve 1 (también si la cola es `NULL`); `front` devuelve `NULL`; `dequeue` avisa y devuelve un enemigo de error con `id = -1`.
Un solo elemento	Al sacarlo, `front` queda en `NULL` y `rear` se reinicia, de modo que la cola se puede volver a usar. Al encolar en una cola vacía, `front` y `rear` apuntan al mismo nodo.
Memoria insuficiente	`create` devuelve `NULL`; `enqueue` devuelve 0 y deja la cola sin cambios. La aventura revisa el resultado de `create`.
Eliminación y liberación	`dequeue` libera el nodo después de copiar al enemigo. `make_empty` guarda el siguiente nodo antes de cada `free`, y la aventura libera la cola al terminar cada zona, incluso si el personaje muere.

Cuadro 7: Casos límite de la cola de enemigos.

6. Guardar y cargar partida

El progreso se guarda en un archivo de texto plano con secciones marcadas, de modo que se pueda leer y depurar a simple vista:

```
RPG_PARTIDA_V1
PERSONAJE
Aria 26 30 8 5 2 0
INVENTARIO
2
3 0 5 Machete
7 2 20 Antiseptico
FIN
```

La línea del personaje contiene nombre, vida actual, vida máxima, ataque, bono de ataque, defensa y bono de defensa. Cada objeto ocupa una línea con id, tipo (el número del `enum`), valor y nombre. Los nombres no pueden contener espacios; `guardar_partida` lo verifica antes de escribir.

Guardar. Se recorre el inventario saltando el centinela y se escribe cada objeto. El contenido se escribe primero en un archivo temporal y solo si todo salió bien se renombra al archivo final, de modo que una partida anterior nunca queda a medias.

Cargar. Todo se lee a variables temporales (un personaje y un inventario nuevos). Cada objeto se agrega con `inventario_agregar`, que detecta IDs duplicados y falta de memoria. Solo si el archivo completo es válido y termina con la marca `FIN` se libera el inventario anterior y se reemplaza por el nuevo; si hay cualquier error, el estado actual del juego queda intacto.

Situación	**Resultado**
Archivo inexistente	`GUARDADO_ERROR_ARCHIVO`
Marca incorrecta, datos incompletos o archivo cortado	`GUARDADO_ERROR_FORMATO`
IDs de objeto repetidos, tipo fuera de rango, vida incoherente	`GUARDADO_ERROR_FORMATO`
Inventario vacío	Se guarda con cantidad 0 y se carga como inventario vacío
`malloc` falla al reconstruir	`GUARDADO_ERROR_MEMORIA`, sin cambios en el juego
Nombre con espacios al guardar	`GUARDADO_ERROR_PARAMETRO`, no se escribe nada

Cuadro 8: Casos manejados al guardar y cargar.

Relación con `cargar_personaje`. El menú inicial ofrece *Nuevo personaje* (`crear_personaje` con valores base) o *Cargar partida* (`cargar_partida`), que restaura el personaje completo (vida actual y bonos de equipamiento) junto con su inventario. La función `cargar_personaje` del módulo `personaje` lee una ficha simple (nombre, vida, ataque y defensa) para crear un personaje nuevo; la interfaz no la usa porque una partida guardada ya contiene al personaje.

7. Ejemplos de ejecución

Las líneas que empiezan con `>` son entradas del usuario y `[...]` indica líneas omitidas. El resto es la salida real del programa.

7.1. Exploración y combate en la Catacumba

```
Entras a la Catacumba...
 [1] antiseptico (poción, 7)
¿Deseas recogerlo? (s/n):
> s
Antiseptico agregado al inventario.
===== COMBATE =====
Aria: 100 PV | Ataque: 15 | Defensa: 5
Enemigo actual: Furtivo (30/30 PV)
Enemigos en espera: 0
> 1
Atacas al Furtivo: 15 de danio | 15/30 pv
¡Tu defensa bloqueo el ataque de Furtivo!
[...]
Atacas al Furtivo: 15 de danio | 0/30 pv
¡Derrotaste al enemigo Furtivo! Se retira de la cola de combate
¡Has derrotado a todos los enemigos. Victoria!
```

7.2. Cola de tres enemigos, equipamiento y Bengala (Páramo)

```
Enemigos: 1. Furtivo (30 PV) 2. Mastin (38 PV) 3. Vandalo (45 PV)
===== COMBATE =====
Enemigo actual: Furtivo (30/30 PV)
Enemigos en espera: 2
> 3 (usar objeto) > 3 (id del arma)
Equipas escopeta. Ataque: 15 -> 30.
> 3 (usar objeto) > 7 (id de la bengala)
Usas bengala: el enemigo se distrae y perdera su proximo ataque
¡El Furtivo queda desorientado y pierde su turno de ataque!
Atacas al Furtivo: 30 de danio | 0/30 pv
¡Derrotaste al enemigo Furtivo! Se retira de la cola de combate
Enemigo actual: Mastin (38/38 PV)
Enemigos en espera: 1
Atacas al Mastin: 30 de danio | 8/38 pv
El Mastin ataca: danio recibido 2 | 98 pv
[...]
¡Derrotaste al enemigo Vandalo! Se retira de la cola de combate
¡Has derrotado a todos los enemigos. Victoria!
```

7.3. Inventario, historial, deshacer y guardado

```
> 3 (usar/equipar) > 4 (id del machete)
Equipas machete. Ataque: 15 -> 25.
> 4 (buscar por nombre) > machete
¡Objeto encontrado! ID: 4 | Nombre: machete | Valor: 10
> 5 (descartar) > 1
Descartaste: antiseptico
> 6 (historial)
- Se descartó un objeto
- Se usó/equipó un objeto
> 7 (deshacer)
Se restauró antiseptico.
> 8 (guardar) > partida.txt
¡Partida guardada con éxito!
```

8. Pruebas

Las pruebas están en la carpeta `test/`. Cada archivo es un programa con su propio `main` y una función auxiliar `verificar(condicion, descripcion)` que cuenta las pruebas ejecutadas y las fallidas e imprime un resumen. `make test` compila cada archivo junto con el código de `src/` usando AddressSanitizer y UndefinedBehaviorSanitizer, que detectan fugas de memoria y accesos inválidos, y ejecuta todos los programas.

- `main_deprueba.c`: módulos `objeto`, `inventario`, `uso_objeto` y `guardado`, con foco en casos límite: estructura vacía, un solo elemento, eliminación del primero, del medio y del último, entradas inválidas y archivos de guardado corruptos.
- `test_estructuras.c`: cola de enemigos y pila del historial (48 comprobaciones): estructura vacía, un solo elemento y reutilización tras vaciarla, orden FIFO y LIFO, conservación del daño al modificar el enemigo del frente, copia del objeto en cada acción, truncado de descripciones largas y punteros `NULL`.

Además se probó el juego completo de forma manual con los escenarios de la Tabla 9.

Escenario	**Resultado observado**
Responder “s” o “n” a “¿Seguir explorando?”	Con “s” continúa la zona; con “n” regresa al menú sin fugas de memoria.
Usar la Bengala en combate	El enemigo pierde su turno de ataque.
Usar la Bengala fuera de combate	No se consume el objeto.
Personaje que muere en combate	La zona termina, se libera la cola y el juego avisa el fin de la partida.
Descartar y deshacer, o deshacer otra acción	Se restaura el objeto; si la última acción no fue un descarte, se informa que no se puede deshacer.
Guardar, cargar en otra ejecución	Se restauran vida, bonos e inventario. Un archivo dañado muestra un error y no cambia la partida.
Entradas no numéricas o fuera de rango	Se muestra un mensaje y se vuelve a pedir la opción.

Cuadro 9: Escenarios probados manualmente.

9. Conclusiones

Implementar las tres estructuras desde cero nos mostró que la elección depende de cómo se accede a los datos. La lista enlazada con nodo centinela resultó la más adecuada para el inventario, donde se busca, usa y elimina en cualquier posición; la pila encaja con un historial que solo se consulta y se deshace por el final; y la cola refleja de forma natural el orden en que se enfrentan los enemigos. El nodo centinela y los punteros de frente y final simplificaron los casos de estructura vacía y de un solo elemento, y guardar copias de los objetos y enemigos evitó problemas de propiedad de la memoria.

También quedaron limitaciones que mejoraríamos con más tiempo: registrar en el historial los eventos del combate, devolver el objeto restaurado a su posición original, y mantener un puntero al último nodo del inventario para agregar en $O(1)$. Las pruebas automáticas y las herramientas de análisis de memoria nos ayudaron a encontrar y corregir errores antes de integrar los módulos.

10. Contribución por integrante

Pablo Serón
Lista enlazada del inventario, módulo `objeto`, uso y equipamiento de objetos (`uso_objeto`), guardar y cargar partida, pruebas de sus módulos y de la cola y la pila, y `Makefile`.

Daniela Soto
Cola de enemigos, atributos del enemigo, módulo de combate y aventura.

Carlos Cofré
Pila del historial y deshacer (`historial`), módulo `personaje` e interfaz de terminal (`interfaz`, menú principal).

11. Declaración de uso de IA

Pablo Serón.

- Herramienta utilizada: Claude (Anthropic).
- Usos: Durante mi desarrollo durante mi parte del proyecto (Estructura Inventario - Módulo Objeto - Usar/equipar objeto - Guardar/Cargar partida), Utilice Claude para la comprensión de errores mediante compilacion de codigo, la correccion de errores mediante la conexion de estos y las identificaciones de errores minusiosos (memory leak).

Daniela Soto.

- Herramienta utilizada: Gemini
- Usos: En combate.c y aventura.c para la correción de errores y el arreglo de warnings arrojado por estas funciones. Y para la creación del nombre de los elementos ocupados como nombre de enemigos, zonas, armas, armaduras y pociones.

Carlos Cofré.

- Herramienta utilizada: Gemini
- Usos: Utilice Gemini principalmente como una herramienta de apoyo para no quedarme pegado. Específicamente lo ocupe para:
  - aclarar algunas dudas conceptuales sobre como armar bien la estructura de la pila con nodos para la parte del historial.
  - ayudarme a pillar y arreglar errores con el manejo de memoria, para asegurarme de que el juego no se cayera ni dejara memoria sin liberar.
  - revisar que el codigo estuviera ordenado y que los comentarios de los archivos .h cumplieran con el formato de buenas practicas.
  - al final, me encargue de leer, probar y entender completamente todo el codigo que entregue para tener super clara la logica de las estructuras que implemente.